RBE549: Project 3 – Einstein Vision – using 5 late days

Nishanth Adithya Chandramouli
Robotics Engineering
Worcester Polytechnic Institute
nchandramouli@wpi.edu

Chaitanya Sriram Gaddipati
Robotics Engineering
Worcester Polytechnic Institute
csgaddipati@wpi.edu

***Abstract*—We present *Einstein Vision*, a Tesla FSD-inspired 3D visualisation pipeline for autonomous driving scenes. Given monocular video from the front camera of a 2023 Tesla Model S, the system detects and tracks vehicles, pedestrians, lane markings, traffic lights, road signs, and miscellaneous objects, then renders a synthetic Blender scene that mirrors the Tesla FSD aesthetic. The pipeline spans three phases: Phase 1 covers detection, depth estimation, and 3D localisation; Phase 2 adds multi-object tracking, pedestrian pose estimation, and fine-grained classification; Phase 3 introduces brake/turn-signal detection, ego-motion-compensated motion classification, and collision-risk prediction. Key contributions include fusing YOLO11 [1] and YOLO-World [2] in a single NMS-unified pass, using CLIP [3] for vehicle sub-type classification, and applying RAFT [6] optical flow with RANSAC epipolar tests to separate independently-moving vehicles from camera ego-motion.**

I. Introduction

Autonomous vehicles must build a rich, real-time model of their surroundings from noisy, partial sensor data. Tesla's Full Self-Driving visualisation distils this model into a synthetic third-person view that conveys depth, motion, and semantic meaning at a glance. Replicating this pipeline requires accurate detection, temporally consistent tracking, metric depth estimation, kinematic state estimation, and high-fidelity 3D rendering in a coherent data flow.

Einstein Vision addresses this challenge. The input is MP4 video from 13 diverse urban scenes; the output is a Blender-rendered video with road surfaces, lane Bézier curves, vehicle and pedestrian meshes at metric 3D positions, traffic signals with correct colour states, and Phase-3 overlays for brake lights, turn signals, motion arrows, and collision-risk highlights.

The pipeline has two stages. The *perception stage* processes each sampled frame to produce a structured `FrameData` record serialised to MessagePack and NumPy archives. The *rendering stage* reads those archives under Blender's embedded Python interpreter and renders each frame to PNG. Both stages are driven by a single YAML configuration file so any model can be swapped without changing pipeline code.

II. Phase 1: Core Perception

A. Object Detection

We fuse two YOLO-family models to cover all required driving-scene classes.

YOLO11 [1] (`yolo11x.pt`) handles the 80 standard COCO classes. We retain only the nine driving-relevant classes: *person, bicycle, car, motorcycle, bus, truck, traffic light, stop sign, fire hydrant*.

YOLO-World [2] (`yolov8x-worldv2.pt`) handles open-vocabulary classes COCO does not cover: *traffic cone, traffic cylinder, dustbin, barrel, speed limit sign, speed bump, and vehicle subtypes (SUV, pickup truck)*. We set a custom class list at runtime via `model.set_classes()`, which inserts a text-embedding branch without retraining.

The two detection lists are concatenated and de-duplicated with a single batched NMS pass (IoU $= 0.45$). Per-class confidence minimums are applied after normalising YOLO-World synonym phrasings (e.g. "road speed bump" $\rightarrow$ "speed bump") so downstream code uses canonical names throughout.

B. Vehicle Sub-classification and Speed-Limit OCR

Vehicle sub-types. YOLO11 reports all light vehicles as `car`. To distinguish sedan / SUV / hatchback / pickup we crop each bounding box and run CLIP [3] ViT-B/32 similarity against four text prompts. Text features are pre-encoded at initialisation; the per-frame cost is one image-encoder forward pass plus a dot product.

Speed limit signs. Detected crops are upscaled to a minimum 160 px short-side, Otsu-thresholded on the upper two-thirds, and passed to EasyOCR with a digit allowlist. If EasyOCR returns a value matching a valid US speed limit $(15, 20, \ldots, 75$ mph), it is recorded and rendered as a texture on the 3D sign asset.

C. Lane Detection

We use a fine-tuned Mask R-CNN [4] (ResNet-50-FPN) with six lane-type output classes (solid-white, dashed-white, solid-yellow, double, etc.). Each binary mask requires at least 150 active pixels; surviving masks are fit with a second-degree polynomial, and 10 Bézier control points are sampled uniformly along each curve. A lightweight IoU-based lane tracker assigns consistent IDs across frames, ageing out unmatched tracks after 10 frames.

D. Traffic Light Classification

Colour. Each bounding-box crop is divided into three equal horizontal strips (top = red, middle = yellow, bottom = green).

The strip with the highest mean HSV V-channel brightness above a threshold is taken as the active colour. This is more robust than hue-range matching under non-neutral lighting.

Arrow direction. The active colour strip is HSV-masked, morphologically cleaned, and its largest contour extracted. A circularity test rejects round bulbs; non-circular contours are classified as *straight*, *left*, or *right* from the principal-axis angle of the contour moments.

E. Depth Estimation and 3D Localisation

Monocular depth. We use Depth Anything V2 Metric Outdoor (Large) [5] via HuggingFace Transformers. It produces float32 metric depth maps at native resolution. Frames are batched on the GPU to amortise startup overhead.

Lifting detections to 3D. Depth is sampled as the median of a 9×9 pixel patch centred on the bounding-box bottom edge (ground contact point). Patch-median is preferred over a single-pixel lookup because depth models are inaccurate at object boundaries. If the sampled value is below 1 m, a perspective-height fallback uses the known real-world class height and pixel height to estimate depth. The pixel is then back-projected through the camera intrinsics into ego frame.

Ground-level classes (vehicles, pedestrians, cones, etc.) use Inverse Perspective Mapping (IPM) instead: assuming a flat ground plane at the camera calibration height, each pixel maps directly to a $(X, Y, 0)$ ego-frame coordinate that is immune to depth-map overestimation at the bbox bottom edge.

Yaw estimation. Vehicle heading is estimated geometrically: the projected ground width of a vehicle of known dimensions at a measured depth is used to solve for yaw, and the front/rear ambiguity is resolved from the sign of the lateral position.

III. Phase 2: Advanced Features

A. Multi-Object Tracking with ByteTrack

We use ByteTrack [7] for multi-object tracking. Detections from the fused YOLO pipeline are forwarded to the tracker each frame. Unlike SORT-family trackers, ByteTrack uses *every* detection including low-confidence ones in a second association pass, significantly reducing ID switches on partially occluded vehicles. Lost tracks are buffered for 30 frames to handle short occlusions. Consistent track IDs are critical for Phase-3 features: brake-light detection uses a per-track temporal buffer, and the collision predictor maintains a per-track position history.

B. Pedestrian Pose Estimation

Pedestrian pose is estimated with YOLO11x-pose, which predicts the 17 standard COCO body keypoints. The 2D keypoints are lifted to 3D using per-keypoint depth samples from the monocular depth map and back-projected into ego frame. The Blender renderer then drives a capsule stick-figure directly from the 3D joint positions, giving a pose-matched silhouette in the Tesla FSD style without requiring a rigged mesh asset.

C. Traffic Sign Detection and OCR

Stop signs are detected by YOLO11. Speed limit signs are detected by YOLO-World with the label `speed limit sign`, then processed by the EasyOCR pipeline (Section II-B). Confirmed detections are serialised with their 3D ego-frame positions so Blender can spawn the correct sign asset.

D. Ground Arrow Detection

Ground arrow markings (turn arrows, keep-right, etc.) are detected by YOLO-World. PCA on the largest contour of each crop yields two eigenvectors; the primary eigenvector's angle classifies the arrow as *straight*, *left*, or *right*.

E. Miscellaneous Object Detection

Traffic cones, traffic cylinders, dustbins, barrels, fire hydrants, and speed bumps are detected by YOLO-World. Each class has a corresponding 3D mesh asset or procedural geometry. Speed bumps use a procedural semi-cylindrical BMesh geometry (3.5 m wide, 0.10 m tall) placed at $Z = 0$.

IV. Phase 3: Bells and Whistles

A. Brake Light and Turn Signal Detection

Three pixel-density ROIs are extracted from each tracked vehicle bounding box.

Brake lights. The bottom 40% of the bbox is HSV-analysed for bright-red pixels ($H \in [0°, 10°] \cup [160°, 179°]$, $S > 80$, $V > 120$). A brake-on event fires when red-pixel density exceeds 5%.

Turn signals. Left and right 30%-wide vertical strips of the lower 60% of the bbox are analysed for amber ($H \in [8°, 28°]$, $S > 110$, $V > 120$). Turn signals blink, so an 8-frame history per track gates the decision: the signal is active when both the mean density over history $> 2\%$ and at least one frame exceeds the full 4% threshold. This two-condition gate rejects brief reflections (fail condition 1) and faint ambient colour (fail condition 2).

In Blender, active brake lights drive small red emissive cuboids spawned at each tail-light corner position. Turn signals spawn amber cuboids on the active side, blinking at 1.5 Hz.

B. Moving vs. Parked Vehicle Classification

Classifying independently-moving vehicles in the presence of ego-motion is ill-posed for pure flow-magnitude thresholding. We use a two-stage approach.

Stage 1 – background flow subtraction. The frame-wide median optical flow vector is a robust estimate of camera-induced background motion (valid when objects occupy less than 50% of the image). The per-vehicle residual flow is the bbox-median minus the frame-median.

Stage 2 – epipolar consistency test. The fundamental matrix F is estimated from 800 random pixel correspondences via RANSAC. A vehicle is classified as *moving* if *either* its residual flow magnitude exceeds 2 px/frame *or* its Sampson epipolar distance exceeds 4.0. The dual criterion handles vehicles moving at similar speed to ego (small residual, large Sampson) as well as faster-than-ego vehicles (large residual).

Optical flow is computed by RAFT-Large [6] (`raft-things`, 20 iterations) on pairs of consecutive sampled frames.

C. Collision Risk Prediction

The collision predictor maintains a 10-frame position history per confirmed track and fits a linear velocity. Tracks with fewer than 3 entries are skipped. Predicted positions 15 frames ahead ($\approx$ 2 s) are compared for all pedestrian–vehicle pairs: if the predicted 3D distance falls below 5 m, both tracks are flagged. The Blender renderer applies a red emissive overlay to flagged meshes.

V. Pipeline Architecture

A. Data Flow

Figure 1 illustrates the end-to-end data flow. The perception stage (`run_perception.py`) reads frames at a configurable stride, runs all models in sequence, and writes one MessagePack file and one or two NumPy archives (depth, flow) per frame. The rendering stage (`scene_builder.py`) reads those files under Blender's embedded Python interpreter.

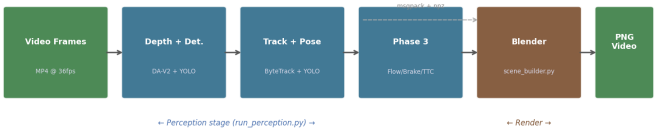

Fig. 1: End-to-end pipeline. Perception (left) produces serialised per-frame records; Blender rendering (right) consumes them headlessly.

B. Per-Frame Processing Order

Within the perception stage, models run in dependency order: (1) depth, (2) detection, (3) sub-classification and OCR, (4) tracking, (5) 3D lifting and yaw, (6) traffic light colour/arrow, (7) lane detection, (8) pedestrian pose, (9) Phase-3 features (brake/turn, optical flow, motion classification, collision prediction).

C. Configuration

Table I summarises the configuration used for all results. All model choices and hyperparameters live in `configs/models.yaml`; swapping a model requires a single YAML edit and no code changes.

TABLE I: Perception pipeline configuration.

Component	Configuration
Depth	Depth Anything V2 Metric Outdoor-L
Detection	YOLO11x + YOLO-World v8x (conf 0.30, IoU 0.45)
Sub-classifier	CLIP ViT-B/32 + EasyOCR
Lane detector	Mask R-CNN ResNet-50-FPN (score 0.50)
Tracker	ByteTrack (lost buffer 30 frames)
Pose	YOLO11x-pose (17 COCO keypoints)
Optical flow	RAFT-Large, `raft-things`, 20 iters
Brake threshold	Red density $> 5\%$
Turn threshold	Amber density $> 4\%$, history 8 frames
Motion: flow	Residual > 2.0 px/frame
Motion: epipolar	Sampson distance > 4.0
Collision horizon	15 frames, warn < 5.0 m

VI. Blender Rendering

Asset placement. The asset manager maintains a library of `.blend` mesh files, one per object class (sedan, SUV, pickup, truck, bus, motorcycle, bicycle, traffic light pole, stop sign, speed limit sign, traffic cone, dustbin, etc.). Vehicle meshes are placed at their 3D ego-frame positions with yaw applied as a Z-axis rotation. Per-track colour assignment ensures the same vehicle carries the same colour throughout, aiding visual continuity.

Lane curves. Lanes are rendered as Bézier curves via Geometry Nodes. Solid and dashed types use different material slots.

Lighting. A world background matched to an overcast sky provides uniform ambient lighting. The light animator spawns small emissive cuboid meshes for brake lights (red) and turn signals (amber) at computed tail-light corner positions, interpolated to match vehicle heading.

Camera. The Blender camera is configured from the calibration YAML (f_x, f_y, c_x, c_y, height, pitch) so the virtual camera matches the real sensor geometry exactly.

VII. Qualitative Results

Figures 2–4 show representative rendered frames from each phase.

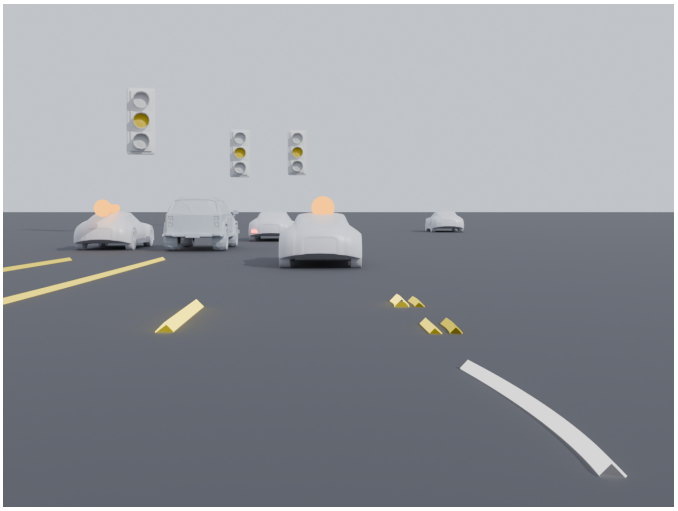

Fig. 2: Phase 1: vehicles placed at metric 3D positions, lane Bézier curves, traffic lights with correct colour states.

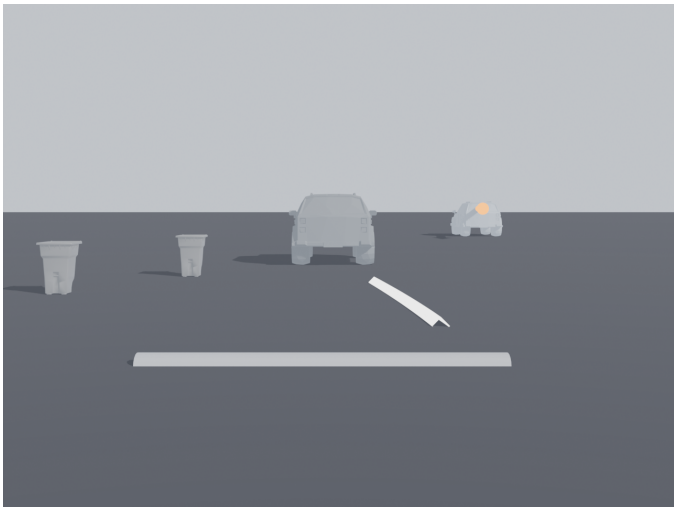

Fig. 3: Phase 2: pedestrian capsule stick-figure driven by COCO 3D keypoints, speed bump semi-cylinder mesh, dustbin assets, SUV at ground level.

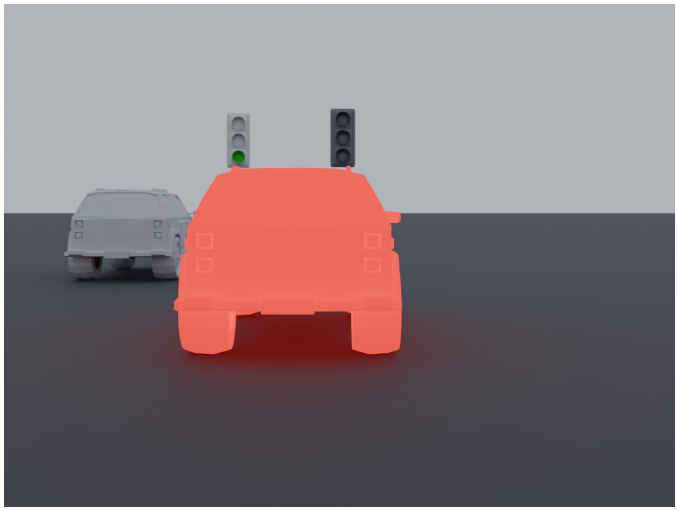

Fig. 4: Phase 3: red emissive collision-risk overlay on the lead vehicle (TTC < 3 s), green traffic light state, dark asphalt ground.

Figure 5 shows the 2D perception debug overlay alongside the Blender render, providing a diagnostic view of detection bounding boxes, track IDs, depth estimates, lane masks, and traffic light states on the original image.

Fig. 5: 2D perception overlay: detection bboxes with class labels, depth, and traffic light state (yellow) on the original camera image.

VIII. Challenges and Failure Modes

Yaw estimation. The geometric yaw estimator assumes a clean rectangular vehicle silhouette. Partially occluded vehicles and tight quarter-views produce yaw errors of 20–40°. A learning-based 3D bounding box estimator would reduce this but requires calibrated depth ground truth.

Depth scale at range. Depth Anything V2 is metric but the absolute scale drifts slightly beyond ≈ 40 m, causing distant vehicles to appear oversized in the render. A two-point

scale calibration using known lane-width measurements would improve consistency.

Traffic light colour at night. Sodium-vapour street lamps elevate the yellow-orange brightness baseline, occasionally causing misclassification of red lights as yellow. A learned detector or per-scene white-balance calibration would address this.

Open-vocabulary false positives. YOLO-World detects speed limit signs with moderate reliability but consistently misidentifies street name signs and ONE WAY signs as speed limit signs. The speed limit confidence threshold (0.45) helps, but real speed limit signs are often too small and distant for reliable OCR.

Lane detection on worn markings. Several scenes contain heavily worn or snow-covered lane markings that Mask R-CNN misses. Pseudo-label fine-tuning on the target domain or a drivable-area segmentation fallback could bridge this gap.

IX. Author Contributions

Nishanth Adithya Chandramouli: Phase-3 motion classification (RAFT integration, Sampson distance, ego-motion compensation), collision predictor, brake/turn detector, Blender rendering pipeline (`scene_builder.py`, `light_animator.py`, `lane_renderer.py`), debug visualisation, ByteTrack integration.

Chaitanya Sriram Gaddipati: Object detection pipeline (YOLO11 + YOLO-World fusion, NMS), vehicle sub-classifier (CLIP + EasyOCR), traffic light HSV classification and arrow detection, depth integration (Depth Anything V2), 3D localisation and yaw estimation, report writing.

References

[1] Ultralytics, "YOLOv11: A state-of-the-art real-time object detection model," 2024. https://docs.ultralytics.com/models/yolo11/

[2] T. Cheng *et al.*, "YOLO-World: Real-Time Open-Vocabulary Object Detection," *arXiv:2401.17270*, 2024.

[3] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," in *Proc. ICML*, 2021.

[4] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. ICCV*, 2017.

[5] Y. Li *et al.*, "Depth Anything V2," *arXiv:2406.09414*, 2024.

[6] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," in *Proc. ECCV*, 2020.

[7] Y. Zhang *et al.*, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *Proc. ECCV*, 2022.

[8] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.